

自动舵控制算法 C++ 代码说明文档

文档编号: AR-CODE-2026-001 版本: V1.0 编制日期: 2026-07-29 对应设计文档: 《自动舵定位与控制设计文档》 § 3~ § 5 代码目录: control/include/

1. 概述

本文档说明自动舵 **控制部分** 的纯算法 C++ 实现，对应设计文档中的：

- § 3.2~ § 3.7 PD 航向控制律
- § 3.6 增益调度
- § 3.9 控制算法标定（Z 形试验辨识 + PD 增益初算）
- § 3.10 控制参数自动优化（RLS 在线辨识 + 极点配置 + ESC）
- § 4 仅航向条件下的机动功能（Leg Sequence 框架）
- § 5 定位与控制联合工作流（主循环集成）

纯算法层：不包含任何硬件 IO（GPIO/CAN/UART/EEPROM），所有传感器输入与舵机输出通过函数参数传递，便于单元测试与 HIL 仿真。

2. 代码结构

```
control/include/
├── angle_utils.hpp      # 角度归一化与差值
├── pd_controller.hpp    # PD 航向控制器
├── gain_schedule.hpp    # 增益调度表
├── nomoto_identifier.hpp # Nomoto 模型 RLS 在线辨识
├── auto_tuner.hpp       # 极点配置 + ESC + 稳定性监控
├── maneuver_sequencer.hpp # 机动航段序列框架
├── param_store.hpp      # 参数存储 + CRC32
└── autopilot_core.hpp   # 核心集成（主循环）
```

所有文件均为 **header-only**，C++17 标准，无第三方依赖，便于嵌入 ECU。

3. 模块详解

3.1 angle_utils.hpp — 角度归一化

对应文档：§ 3.2 角度归一化

函数	说明
<code>normalizeAngleRad(a)</code>	将角度归一化到 $(-\pi, \pi]$ ，弧度
<code>normalizeAngleDeg(a)</code>	将角度归一化到 $(-180, 180]$ ，度
<code>angleDiffRad(ref, act)</code>	最小带符号差 $ref - act$ ，弧度
<code>angleDiffDeg(ref, act)</code>	同上，度

用途：航向误差计算必须做角度归一化，否则 359° 与 1° 的误差会被误算为 358° 而非 -2° 。

```
const double e = angleDiffDeg(headingRef, headingAct); // 落在 (-180, 180]
```

3.2 pd_controller.hpp — PD 航向控制器

对应文档：§ 3.2 控制律、§ 3.3 限幅、§ 3.7 抗噪与死区

控制律

$$\delta_k = K_p, e_k + K_d, \dot{e}_k$$

关键特性

特性	实现位置	说明
角度归一化误差	<code>angleDiffDeg</code>	§ 3.2
微分低通滤波	<code>alpha = 1/(1+τ/dt)</code>	§ 3.7 一阶低通, $\tau=0.2\text{ s}$
误差死区	`	e
舵角限幅	<code>±dMax</code>	§ 3.3, 默认 $\pm 35^\circ$
舵速限幅	<code>ddMax · dt</code>	§ 3.3, 默认 $10^\circ/\text{s}$

特性	实现位置	说明
输出量化	<code>round(v/quant) * quant</code>	§ 3.7, 默认 0.1°

使用

```

ar::PdController::Params p;
p.Kp = 1.5; p.Kd = 0.6;
p.dt = 0.02; // 50 Hz
ar::PdController pd(p);

double rudderCmd = pd.update(headingRef, headingAct);

```

参数说明

参数	默认值	单位	说明
Kp	1.0	—	比例增益
Kd	0.5	—	微分增益
dMax	35.0	deg	舵角限幅
ddMax	10.0	deg/s	舵速限幅
deadzone	0.3	deg	误差死区
tau	0.2	s	微分滤波时间常数
quant	0.1	deg	输出量化步长
dt	0.02	s	控制周期

3.3 gain_schedule.hpp — 增益调度

对应文档：§ 3.6 / § 3.10.4

按船速与海况查询 PD 增益，内置默认调度表（与文档 § 3.10.4 一致）：

船速区间	海况	Kp	Kd
< 4 kn	0~1	2.0	0.8

船速区间	海况	Kp	Kd
4~10 kn	0~1	1.5	0.6
4~10 kn	2~3	1.0	0.4
10~18 kn	0~1	1.2	0.5
10~18 kn	2~3	0.9	0.4
18~25 kn	0~1	1.0	0.4
18~25 kn	2~3	0.7	0.3

```
ar::GainSchedule sched;  
double Kp, Kd;  
sched.query(speedKn, seaState, Kp, Kd);
```

3.4 nomoto_identfier.hpp — Nomoto 模型在线辨识

对应文档：§ 3.9.4 Z 形试验辨识 / § 3.10.3 RLS 在线辨识

模型

$$T\dot{r} + r = K\delta; \Rightarrow; r_k = K\delta_k - \frac{T}{\Delta t}(r_k - r_{k-1})$$

RLS 递推

$$K_k = \frac{P_{k-1}\phi_k}{\lambda + \phi_k^T P_{k-1} \phi_k}$$

$$\hat{\theta}_k = \hat{\theta}_{k-1} + K_k(y_k - \phi_k^T \hat{\theta}_{k-1})$$

$$P_k = \frac{1}{\lambda}(P_{k-1} - K_k \phi_k^T P_{k-1})$$

其中 $\theta = [K, T]^T$, $\phi_k = [\delta_k, -(r_k - r_{k-1})/\Delta t]^T$, $y_k = r_k$ 。

激励条件

仅当 $|\delta| > 3^\circ$ 且 $|r| > 0.5^\circ/s$ 时更新，避免无激励时的协方差爆炸。

使用

```
ar::NomotoIdentifier id;
id.update(rudderDeg, rateDegS);    // 每 1 s 调用一次
double K = id.K(), T = id.T();
```

3.5 auto_tuner.hpp — 控制参数自动优化

对应文档：§ 3.10

三层架构

层	方法	函数
层 1 在线辨识	RLS	NomotoIdentifier
层 2 极点配置	闭环 ζ/ω_n 反推	AutoTuner::polePlacement
层 3 性能寻优	ESC 极值搜索	AutoTuner::escStep

极点配置（层 2）

$$K_p = \frac{\omega_n^2 T}{K}, \quad K_d = \frac{2\zeta\omega_n T - 1}{K}$$

```
double Kp, Kd;
AutoTuner::polePlacement(K, T, 0.85, 0.20, Kp, Kd);
```

性能指标（ITAE）

$$J = \int_0^{T_w} \left(t|e| + w_u|\delta| + w_{du}|\dot{\delta}| \right) dt$$

ESC 更新律

$$K_{p,k+1} = K_{p,k} - \eta, \widehat{\partial J / \partial K_p}$$

通过对 K_p, K_d 加正弦扰动（相位错开 90° 避免耦合），解调估计梯度。

稳定性监控（§ 3.10.7）

```
bool ok = tuner.stabilityCheck(K, T, Kp, Kd, oscAmplitude);
// 四条件:  $\zeta \geq 0.6$ ,  $\omega_n \leq 0.5$ , 振荡  $\leq 5^\circ$ , 增益变化  $\leq 10\%$ 
```

安全约束

约束	阈值
单次更新幅值	$\leq 10\%$
高速去激活	$> 15\text{ kn}$
稳定性四条件	全满足才接受
回退	<code>revert()</code> 恢复基线

3.6 maneuver_sequencer.hpp — 机动航段序列

对应文档：§ 4.2 通用机动调度框架

Leg 结构

```
struct Leg {
    Mode    mode;        // HEADING / RUDDER
    double  target;      // 目标航向 或 目标舵角
    Trigger trigger;     // HEADING_REACHED / TIME / RATE / NONE
    double  threshold;   // 触发阈值
    int     next;        // 下一段索引, -1 结束
};
```

预置机动

函数	对应文档	说明
<code>williamson(psi0)</code>	§ 4.4	$+\delta_{\text{max}}$ 至 $\Delta\psi=60^\circ \rightarrow -\delta_{\text{max}}$ 至 $\Delta\psi=180^\circ \rightarrow$ 保持
<code>uTurn(psi0)</code>	§ 4.5	PD 跟踪 $\psi_0 \pm 180^\circ$
<code>zigzag(psi0)</code>	§ 4.6	$\pm\delta_z$ 与 $\pm\phi_z$ 交替 N 次
<code>circles()</code>	§ 4.3	恒舵角至累计 360°

函数	对应文档	说明
<code>cloverleaf(psi0)</code>	§ 4.7	4×(转向 270° + 直航 t_leg)
<code>search(psi0)</code>	§ 4.8	扩张方搜，边长递增

使用

```
auto legs = ar::ManeuverSequencer::williamson(psi0);
ar::ManeuverSequencer seq;
seq.setLegs(legs);
while (!seq.done()) {
    auto cmd = seq.update(headingAct, rateAct, dt);
    // cmd.isHeading ? PD 跟踪 cmd.heading : 直接输出 cmd.rudder
}
```

3.7 param_store.hpp — 参数存储

对应文档：§ 3.9.8 / § 3.10.9

ControlParams 结构

字段	对应 EEPROM 区块
rudderZero/Left/Right, ddotMax	区块 A
K, T	区块 B
Kp, Kd	区块 C
version, timestamp, crc	元信息

CRC32

采用 IEEE 802.3 多项式 `0xEDB88320`，校验除 crc 字段外的全部内容。

IO 注入

```
ar::ParamStore store(readFn, writeFn); // 由 BSP 注入
ar::ControlParams p;
if (!store.load(p)) { /* CRC 失败, 加载默认 */ }
p.Kp = 1.5;
store.save(p); // 自动 version+1 + CRC
```

3.8 autopilot_core.hpp — 核心集成

对应文档：§ 5 定位与控制联合 workflow

模式

模式	说明
MANUAL	透传舵角反馈
AUTO_HEADING	航向保持 (PD + 增益调度 + 自动优化)
AUTO_MANEUVER	机动序列执行
HOLD	保持当前舵角

主循环

```
ar::AutopilotCore ap;
ap.setMode(ar::Mode::AUTO_HEADING);
ap.setHeadingRef(128.5);

while (running) {
    ar::SensorInput s = readSensors();
    double rudderCmd = ap.step(s, 0.02);
    writeRudder(rudderCmd);
}
```


主循环内部流程

```
step(s, dt):
  |— MANUAL: 透传
  |— AUTO_HEADING:
    |   |— 增益调度查询 Kp/Kd
    |   |— 自动优化（高速去激活）
    |   |   |— RLS 在线辨识 K/T
    |   |   |— 极点配置重算
    |   |   |— 稳定性监控 → 接受/回退
    |   |— 累积性能指标样本
    |   |— PD 控制律 → 目标舵角
  |— AUTO_MANEUVER:
    |   |— 序列推进
    |   |— HEADING 段走 PD / RUDDER 段直接输出
  |— HOLD: 输出 0
```

4. 数据流与接口

4.1 输入

```
struct SensorInput {
    double headingDeg;    // AHRS 艏向
    double rateDegS;      // AHRS 转向率
    double rudderDeg;     // 舵角反馈
    double speedKn;       // 船速
    int    seaState;      // 海况 0..3
};
```

4.2 输出

目标舵角（deg），由外部驱动层转换为 PWM/CAN 指令。

4.3 调用周期

模块	周期	说明
pd_controller	20 ms (50 Hz)	主控制环
nomoto_identifier	1 s	RLS 更新
auto_tuner.accumulate	20 ms	性能指标采样

模块	周期	说明
auto_tuner.escStep	60 s (窗口)	ESC 微调
maneuver_sequencer	20 ms	段推进

5. 与设计文档的映射

设计文档章节	代码模块	关键函数
§ 3.2 控制律	pd_controller	update()
§ 3.3 限幅	pd_controller	内部 clip
§ 3.6 增益调度	gain_schedule	query()
§ 3.7 抗噪死区	pd_controller	滤波/死区/量化
§ 3.8 稳定性分析	auto_tuner	stabilityCheck()
§ 3.9.4 Z 形辨识	nomoto_identifier	update()
§ 3.9.5 PD 增益初算	auto_tuner	polePlacement()
§ 3.9.8 参数存储	param_store	save()/load()
§ 3.10.3 RLS 辨识	nomoto_identifier	update()
§ 3.10.4 极点配置	auto_tuner	polePlacement()
§ 3.10.5 ESC 寻优	auto_tuner	escStep()
§ 3.10.6 触发逻辑	autopilot_core	step() 内分支
§ 3.10.7 稳定性监控	auto_tuner	stabilityCheck()/revert()
§ 4.2 机动框架	maneuver_sequencer	setLegs()/update()
§ 4.3~ § 4.8 预置机动	maneuver_sequencer	williamson()/uTurn()/...
§ 5 联合工作流	autopilot_core	step()

6. 编译与测试

6.1 编译要求

- C++17
- 无第三方依赖
- header-only，直接 `#include` 即可

6.2 单元测试建议

模块	测试要点
angle_utils	跨 0°/360° 边界误差
pd_controller	阶跃响应、限幅、死区
gain_schedule	跨段查询、海况限幅
nomoto_identifier	已知 K/T 数据收敛性
auto_tuner	极点配置精度、ESC 收敛
maneuver_sequencer	各预置机动段推进
param_store	CRC 校验、版本递增

6.3 HIL 集成

`AutopilotCore::step()` 是唯一入口，注入 `SensorInput`、输出目标舵角，与船舶六自由度模型闭环即可进行 HIL 仿真。

7. 安全约束实现汇总

约束	实现位置	说明
舵角限幅 $\pm 35^\circ$	<code>PdController::update</code>	§ 3.3
舵速限幅 $10^\circ/\text{s}$	<code>PdController::update</code>	§ 3.3
误差死区 0.3°	<code>PdController::update</code>	§ 3.7

约束	实现位置	说明
微分低通 $\tau=0.2\text{ s}$	<code>PdController::update</code>	§ 3.7
RLS 激励门限	<code>NomotoIdentifier::update</code>	§ 3.10.3
单次更新 $\leq 10\%$	<code>AutoTuner::clampUpdate</code>	§ 3.10.7
高速去激活 $> 15\text{ kn}$	<code>AutoTuner::enable</code>	§ 3.10.6
稳定性四条件	<code>AutoTuner::stabilityCheck</code>	§ 3.10.7
回退基线	<code>AutoTuner::revert</code>	§ 3.10.7
参数 CRC32	<code>ParamStore::save/load</code>	§ 3.9.8
版本号递增	<code>ParamStore::save</code>	§ 3.10.9
模式切换	<code>AutopilotCore::setMode</code>	§ 5.2

8. 扩展接口

扩展点	接口	说明
自定义机动	<code>ManeuverSequencer::setLegs</code>	注入 Leg 序列
自定义增益调度	<code>GainSchedule</code> 构造	替换默认表
EEPROM IO	<code>ParamStore(ReadFn, WriteFn)</code>	注入 BSP 读写
性能指标权重	<code>AutoTuner::Params</code>	调整 ITAE 权重
极点配置目标	<code>AutoTuner::Params.zeta/wn</code>	按船速分段
控制周期	<code>PdController::Params.dt</code>	适配不同 ECU